

CSE_2323

K MAP

Simplifying logic circuits

- Two methods for simplifying logic circuits

1. Utilizing Boolean algebra

- Dependent on inspiration and experience
- No unique simplified expression
- Process of trial and error

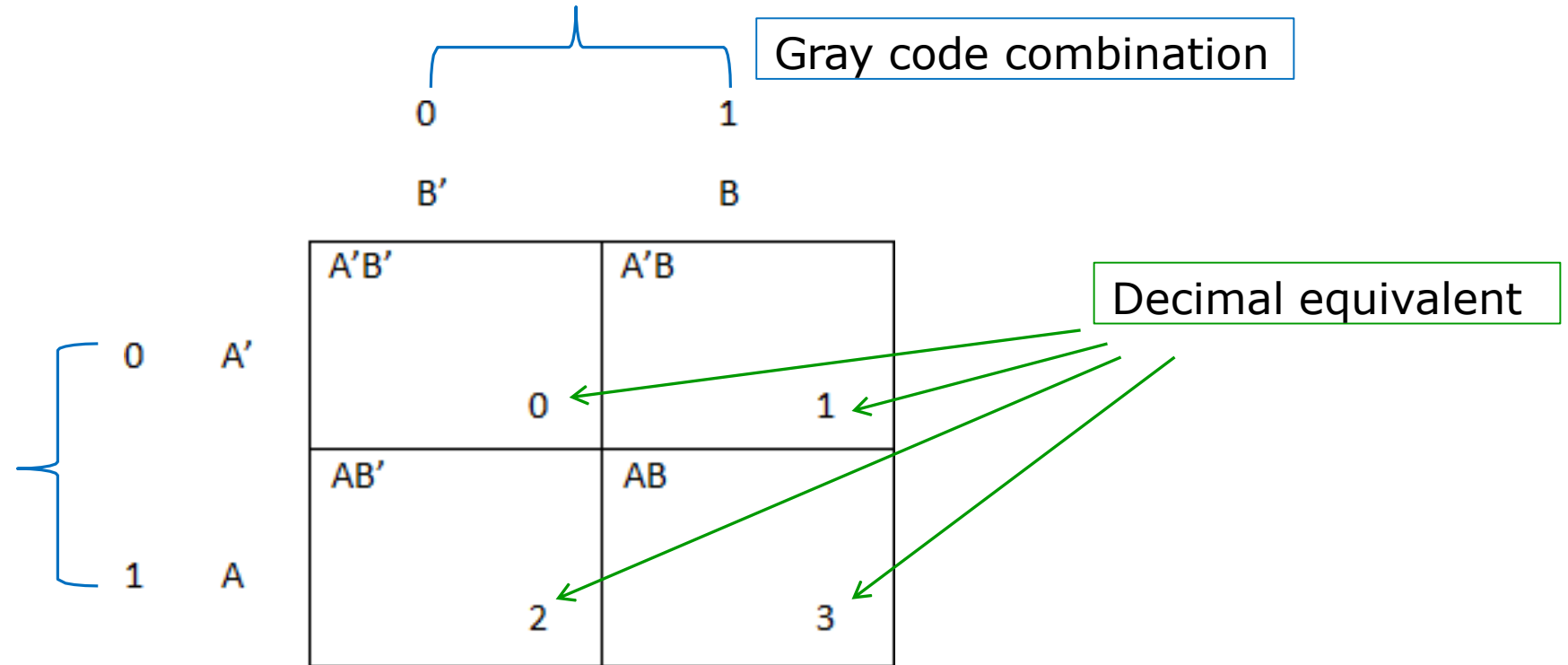
2. Karnaugh mapping

- Systemic and step-by-step approach

Two Variables K-Map

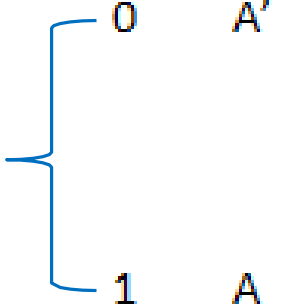
- ❑ Two variables form four minterms
- ❑ So, the map consists of four squares, one of each minterm.

m_0	m_1
m_2	m_3



Three Variables K-Map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

		00	01	11	10
		$B'C'$	$B'C$	BC	BC'
	0 A'	$A'B'C'$ 0	$A'B'C$ 1	$A'BC$ 3	$A'BC'$ 2
	1 A	$AB'C'$ 4	$AB'C$ 5	ABC 7	ABC' 6

Decimal equivalent

Four Variables K-Map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

		00	01	11	10
		$C'D'$	$C'D$	CD	CD'
00	$A'B'$	$A'B'C'D'$ 0	$A'B'C'D$ 1	$A'B'CD$ 3	$A'B'CD'$ 2
		$A'BC'D'$ 4	$A'BC'D$ 5	$A'BCD$ 7	$A'BCD'$ 6
01	$A'B$	$ABC'D'$ 12	$ABC'D$ 13	$ABCD$ 15	$ABCD'$ 14
		$AB'C'D'$ 8	$AB'C'D$ 9	$AB'CD$ 11	$AB'CD'$ 10
11	AB				
10	AB'				

Four Variables K-Map

A=0

		00	01	11	10
		D'E'	D'E	DE	DE'
00	B'C'	B'C'D'E' 0	B'C'D'E 1	B'C'DE 3	B'C'DE' 2
01	B'C	B'CD'E' 4	B'CD'E 5	B'CDE 7	B'CDE' 6
11	BC	BCD'E' 12	BCD'E 13	BCDE 15	BCDE' 14
10	BC'	BC'D'E' 8	BC'D'E 9	BC'DE 11	BC'DE' 10

Five Variables K-Map

A=1

		00	01	11	10
		D'E'	D'E	DE	DE'
00	B'C'	B'C'D'E' 16	B'C'D'E 17	B'C'DE 19	B'C'DE' 18
01	B'C	B'CD'E' 20	B'CD'E 21	B'CDE 23	B'CDE' 22
11	BC	BCD'E' 28	BCD'E 29	BCDE 31	BCDE' 30
10	BC'	BC'D'E' 24	BC'D'E 25	BC'DE 27	BC'DE' 26

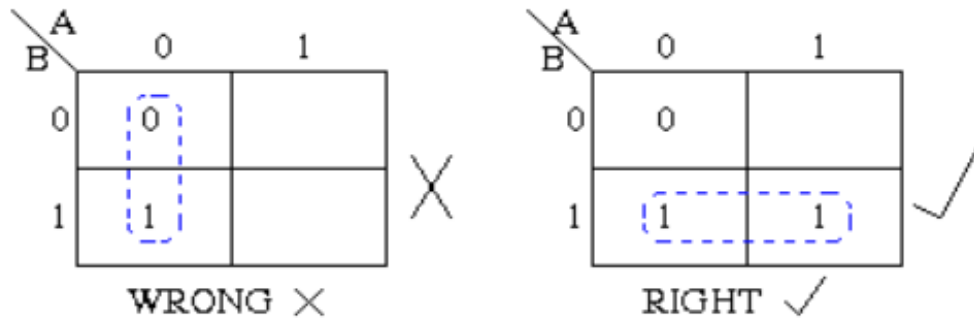
Rules of simplifying K Map

Check adjacent "1" to make

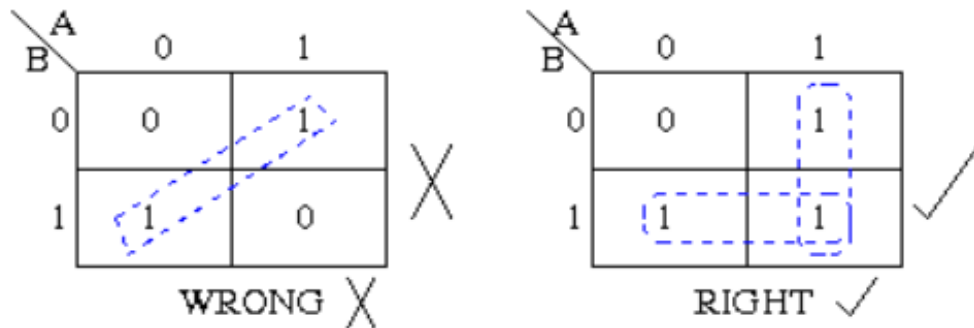
- i. Octet (eight "1")*
- ii. Quad (four "1")*
- iii. Pair (two "1")*
- iv. Isolated "1"*

Rules of simplifying K Map

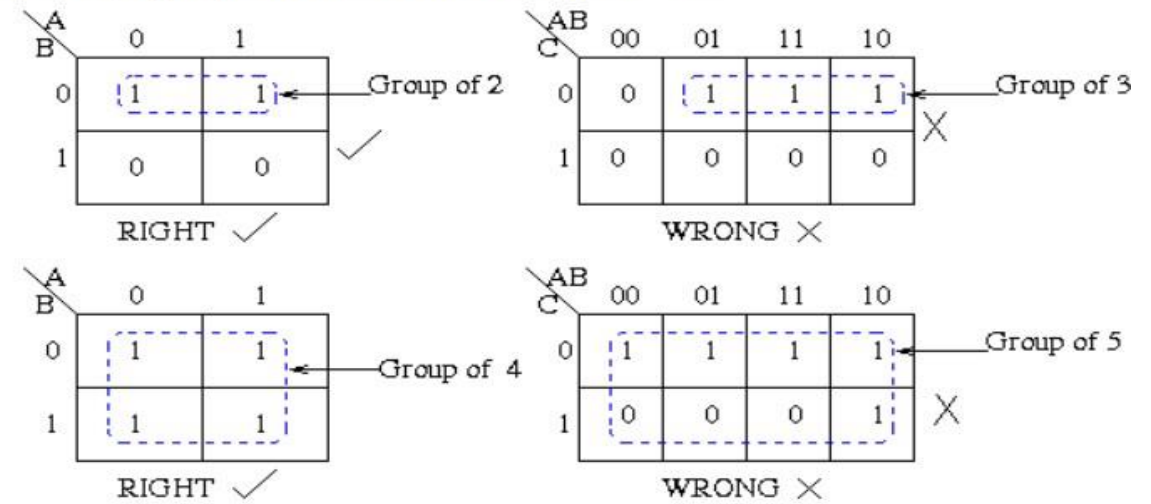
- Groups may not include any cell containing a **zero**



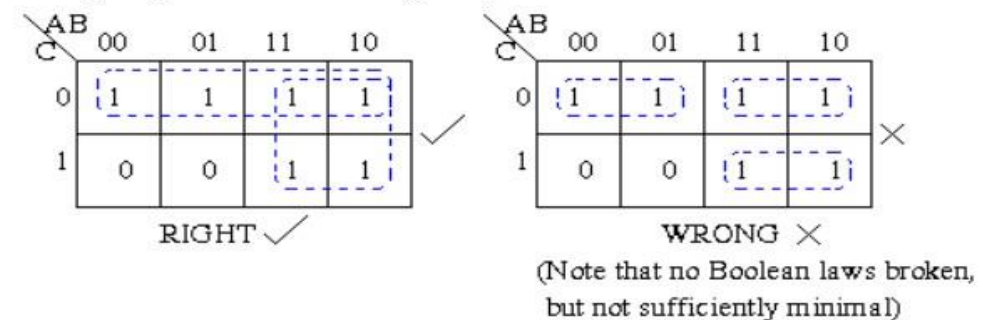
- Groups may be horizontal or vertical, but not diagonal.



- Groups must contain 1, 2, 4, 8, or in general 2^n cells.
That is if $n = 1$, a group will contain two 1's since $2^1 = 2$.
If $n = 2$, a group will contain four 1's since $2^2 = 4$.

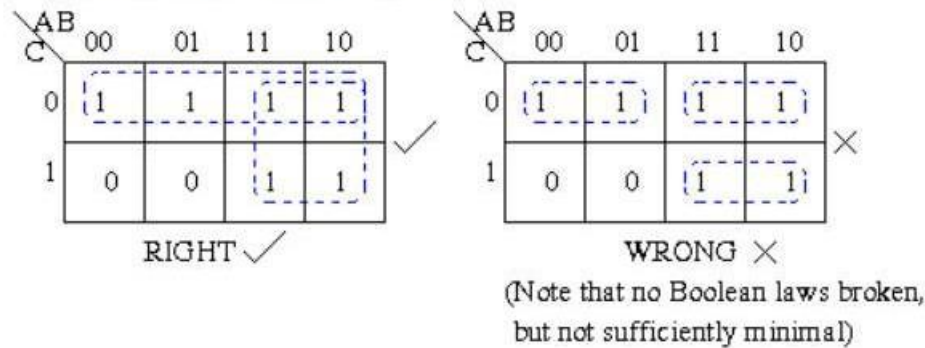


- Each group should be as large as possible.

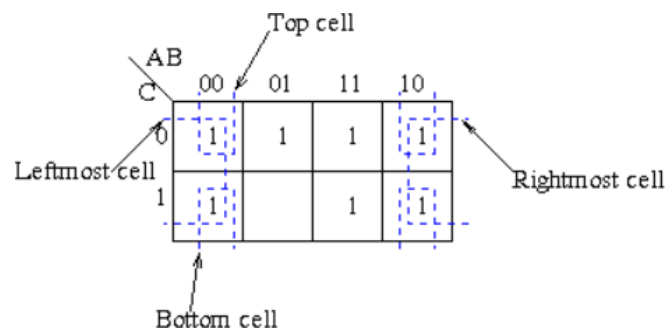
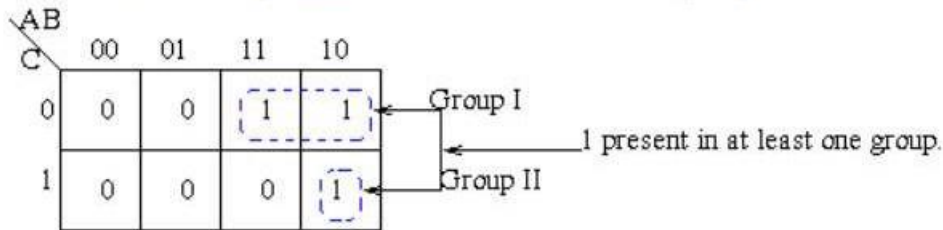


Rules of simplifying K Map

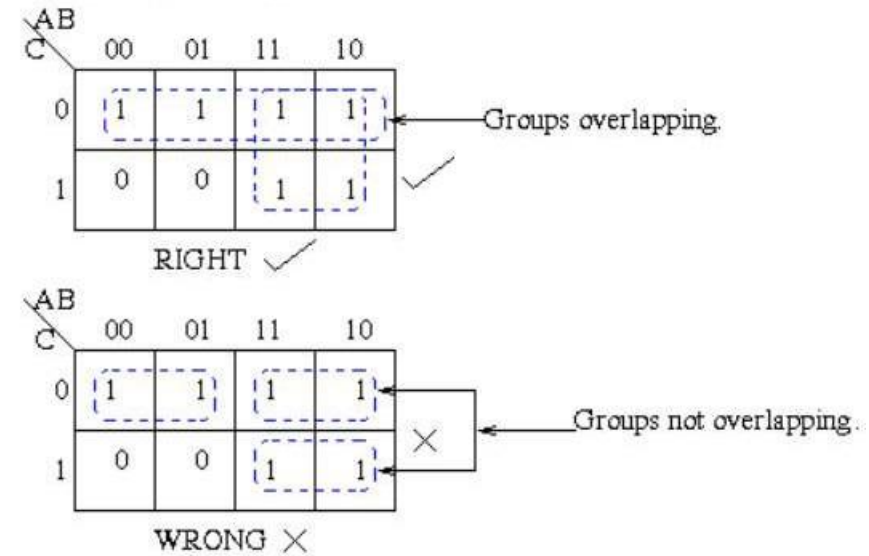
- Each group should be as large as possible.



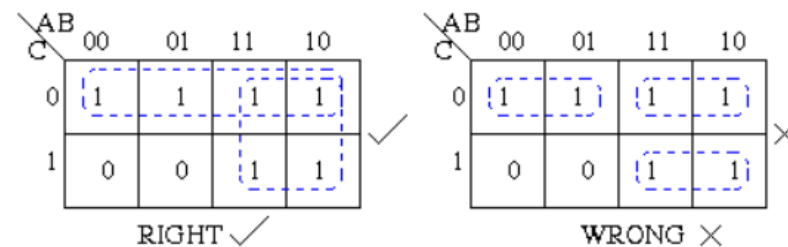
- Each cell containing a **one** must be in at least one group.



- Groups may overlap.



- There should be as few groups as possible, as long as this does not contradict any of the previous rules.



Sum of Products simplification using K-map ...

□ Example-2: Simplify the Boolean function $F(A,B,C) = \sum (0,1,2,3,4,6,7)$

	$B'C'$	$B'C$	BC	BC'
A'	1 0	1 1	1 3	1 2
A	1 4	0 5	1 7	1 6

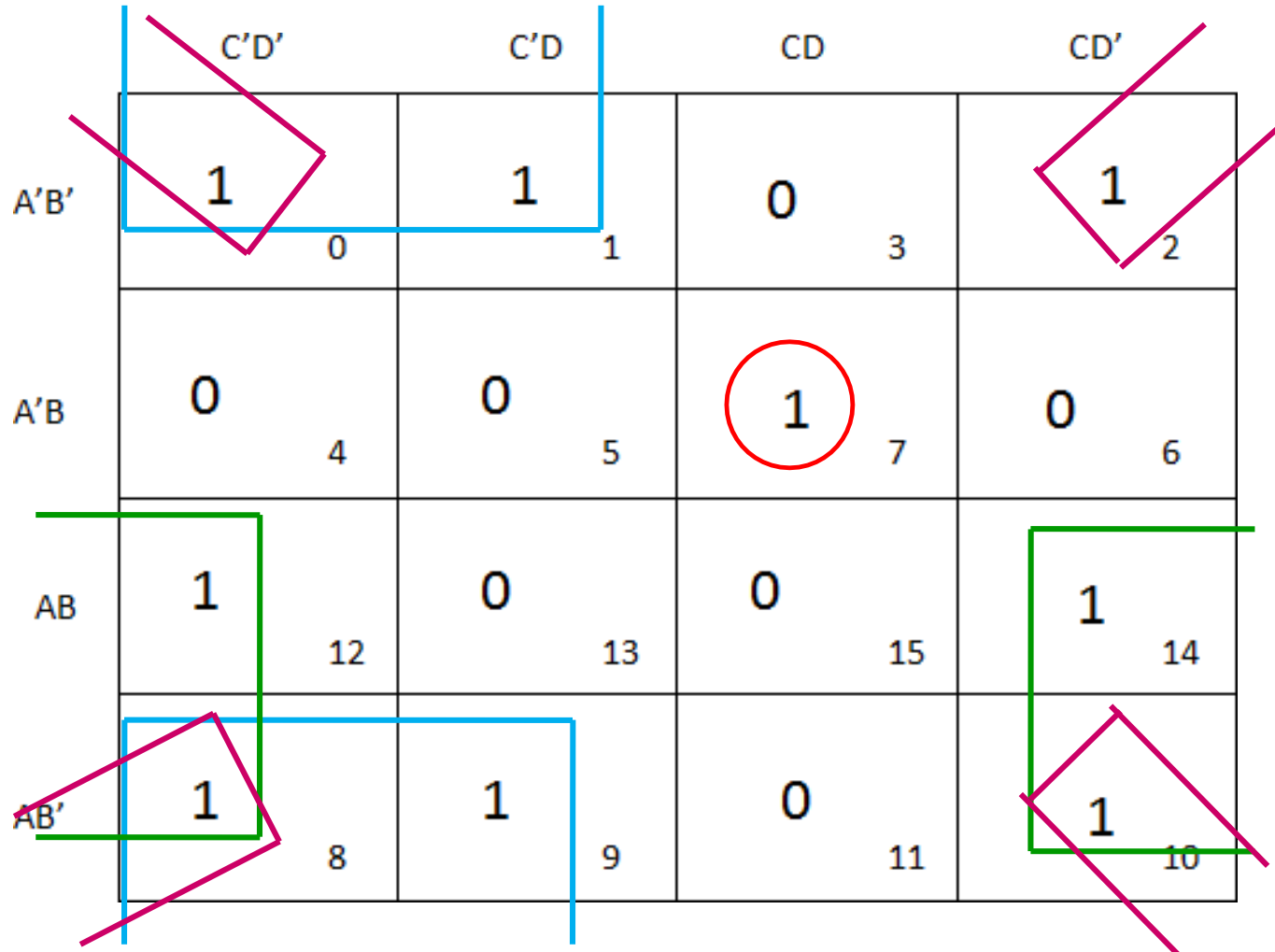
Simplified function $F = A' + C' + B$

Test: Simplify the Boolean function $F(A,B,C) = \sum (0,1,3,7)$

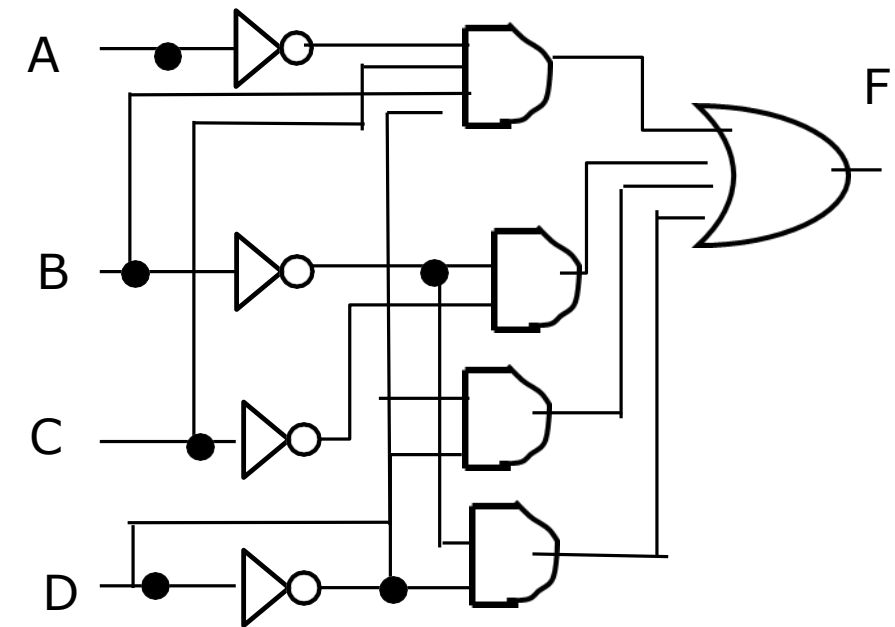
Ans. $F = A'B' + BC$

Sum of Products simplification using K-map ...

❑ Example-3: Given $F(A,B,C,D) = \Sigma(0,1,2,7,8,9,10,12,14)$. Simplify function and implement that function in minimum no. of gates.



$$F = B'C' + AD' + B'D' + A'BCD$$



Sum of Products simplification using K-map ...

□ Example-4: Given $F(A,B,C,D,E) = \Sigma(0,2,4,6,9,13,21,23,25,29,31)$

	D'E'	D'E	DE	DE'
B'C'	1 0			1 2
B'C	1 4			1 6
BC		1 13		
BC'		1 9		

	D'E'	D'E	DE	DE'
B'C'				
B'C		1 21	1 23	
BC		1 29	1 31	
BC'		1 25		

A=0

$$F = A(CE + BD'E) + A'(B'E' + BD'E)$$

A=1

$$= ACE + ABD'E + A'B'E' + A'BD'E$$

$$= A'B'E' + BD'E(A + A') + ACE = A'B'E' + BD'E + ACE$$

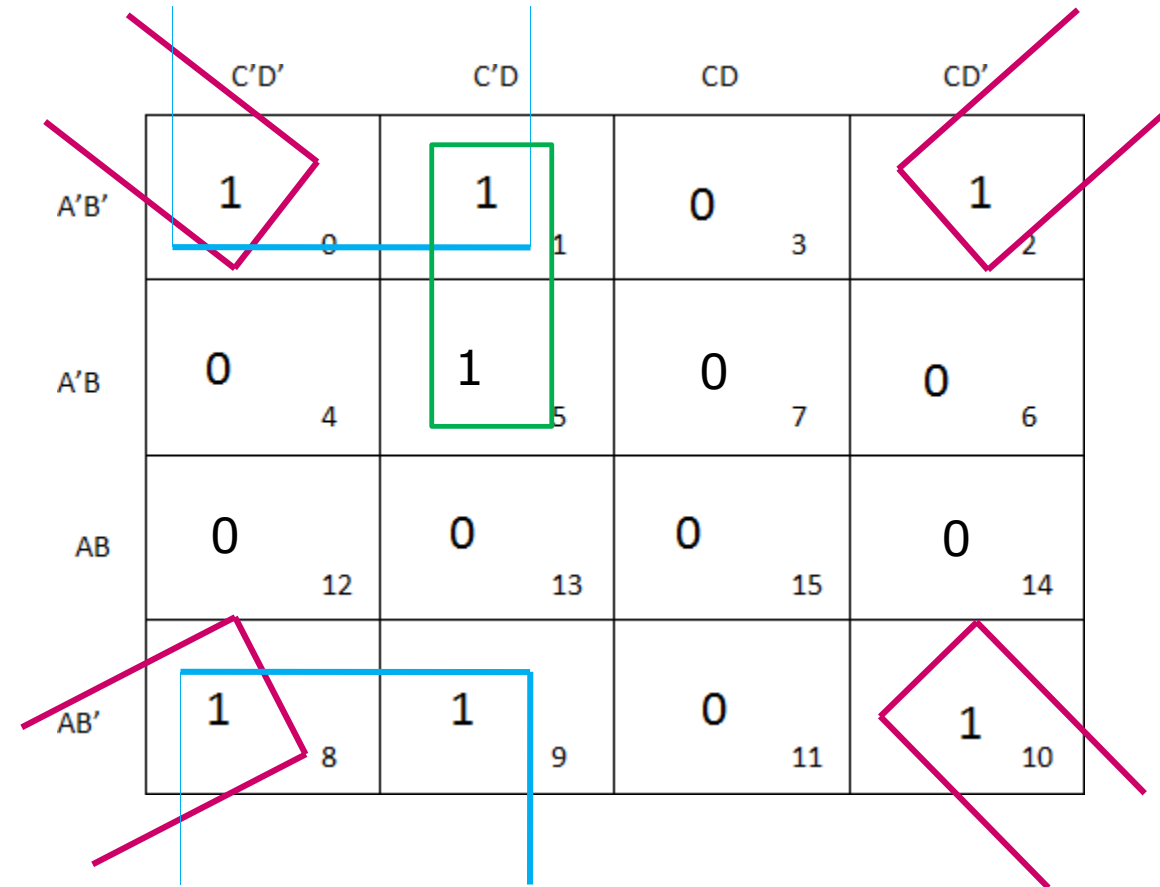
Simplification using K-map ...

❑ Example-1: Simplify $F(A,B,C,D) = \Sigma(0,1,2,5,8,9,10)$.

❖ For SOP representation (previously done)

- Looping '1's

$$F = B'C' + B'D' + A'C'D$$



Product of Sums (POS) Simplification using K-map ...

❑ Example-2: Simplify the Boolean function $F(A,B,C,D)=\Pi(0,1,2,3,4,10,11)$

$$F' = B'C + A'B' + A'C'D'$$

$$F = (F')' = (B'C + A'B' + A'C'D')'$$

$$= (B'C)' (A'B')' (A'C'D')'$$

$$= (B+C') (A+B) [(A')' + (C')' + (D')']$$

$$= (B+C') (A+B) (A+C+D)$$

	C'D'	C'D	CD	CD'
A'B'	0 0	0 1	0 3	0 2
A'B	0 4	1 5	1 7	1 6
AB	1 12	1 13	1 15	1 14
AB'	1 8	1 9	0 11	0 10

Product of Sums (POS) Simplification using K-map ...

❑ Example-2: Simplify the Boolean function $F(A,B,C,D)=\Pi(0,1,2,3,4,10,11)$

$$F' = B'C + A'B' + A'C'D'$$

$$F = (F')' = (B'C + A'B' + A'C'D')'$$

$$= (B'C)' (A'B')' (A'C'D')'$$

$$= (B+C') (A+B) [(A')' + (C')' + (D')']$$

$$= (B+C') (A+B) (A+C+D)$$

	C'D'	C'D	CD	CD'
A'B'	0 0	0 1	0 3	0 2
A'B	0 4	1 5	1 7	1 6
AB	1 12	1 13	1 15	1 14
AB'	1 8	1 9	0 11	0 10

Product of Sums (POS) Simplification using K-map ...

Simplify the expression using MAP $F = (A+B'+C).(A' + B).(B' + C + D')(A'+B'+C+D)$

Do Yourself

Don't Care!

- ❑ In truth table, a certain combination that defines a function represented by '1', and '0' for other conditions.
- ❑ This is not always true
 - There are applications where certain combinations of input variables never occur.
 - e.g., 4-bit decimal code (6-combinations not used)
- ❑ Don't always need all 2^n input combinations in an n -variable function.
 - ✓ If guaranteed that certain input combinations never occur.
 - ✓ If some outputs aren't used in the rest of the circuit.
- ❑ Don't-care outputs marks in truth tables and K-maps with X (read as *cross*).
 - ❑ Within a K-map, each X can be considered as either 0 or 1.
 - Pick the interpretation that allows for the most simplification.

Don't Care Conditions

- ❑ A don't care condition marked by X in truth table, indicates a condition where the design doesn't care if the output is a 0 or 1.
- ❑ X can be treated as a 0 or 1 in a K-Map.
 - Treating X as 0 means do not need to group it.
 - Treating X as a 1 allows to make a grouping larger, resulting in a simpler term in the SOP equation.

x	y	z	f(x,y,z)
0	0	0	0
0	0	1	1
0	1	0	X
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	X
1	1	1	1

Don't Care Conditions...

- ❑ *Some You Group, Some You Don't.*

This don't care condition was treated as a 1. This allowed the grouping of a single one to become a grouping of two, resulting in a simpler term.

$$F = AB'$$

	B'C'	B'C	BC	BC'
A'	0	0	0	0
A	X	1	0	X

There was no advantage in treating this don't care condition as a 1, thus it was treated as a 0 and not grouped.

Don't Care Conditions...

❑ Example-3: Simplify $F(A,B,C,D) = \sum(1,3,7,11,15)$ which has the don't care conditions $d(A,B,C,D) = \sum(0,2,5)$.

❖ SOP form

✓ (Looping 1's and some X's)

$$F = A'B' + CD$$

(*X* considered as 0)

	C'D'	C'D	CD	CD'
A'B'	X 0	1 1	1 3	X 2
A'B	0 4	X 5	1 7	0 6
AB	0 12	0 13	1 15	0 14
AB'	0 8	0 9	1 11	0 10

Don't Care Conditions... (Example-3)

❑ Example-3: Simplify $F(A,B,C,D) = \sum(1,3,7,11,15)$ which has the don't care conditions $d(A,B,C,D) = \sum(0,2,5)$.

❖ SOP form

✓ (Looping 1's and some X's)

$$F = A'D + CD$$

(*X* considered as 0)

	C'D'	C'D	CD	CD'
A'B'	X 0	1 1	1 3	X 2
A'B	0 4	X 5	1 7	0 6
AB	0 12	0 13	1 15	0 14
AB'	0 8	0 9	1 11	0 10

Don't Care Conditions... (Example-4)

- Example-4: Implement the following function with NAND gate.

$$F = A'BD + A'CD + B'CD' + ABC'D \text{ and } d = ACD$$

Class Work

	C'D'	C'D	CD	CD'
A'B'	0 0	0 1	<u>1</u> 3	1 2
A'B	0 4	1 5	1 7	0 6
AB	0 12	1 13	X 15	0 14
AB'	0 8	0 9	<u>X</u> 11	<u>1</u> 10

Assignment

3-1 Simplify the following Boolean functions using three-variable maps:

$$) F(x, y, z) = \Sigma(3, 5, 6, 7)$$

$$) F(A, B, C) = \Sigma(0, 2, 3, 4, 6)$$

3-2 Simplify the following Boolean expressions using three-variable maps:

$$(c) A'B + BC' + B'C'$$

3-3 Simplify the following Boolean functions using four-variable maps:

$$(a) F(A, B, C, D) = \Sigma(4, 6, 7, 15)$$

$$(b) F(A, B, C, D) = \Sigma(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15)$$

$$(c) F(A, B, C, D) = \Sigma(3, 7, 11, 13, 14, 15)$$

3-5 Simplify the following Boolean expressions using four-variable maps:

$$(a) A'B + A'BC + A'BCD + A'BCD'$$

$$(b) B'D + A'BC' + AB'C + ABC'$$

$$(c) AB'C + B'C'D' + BCD + ACD' + A'B'C + A'BC'D$$

3-8 Simplify the following Boolean functions using five-variable maps:

$$(b) F(A, B, C, D, E) = \Sigma(0, 2, 3, 4, 5, 6, 7, 11, 15, 16, 18, 19, 23, 27, 31)$$

$$(c) F = A'B'CE' + A'B'C'D' + B'D'E' + B'CD' + CDE' + BDE'$$

Last Date of Submission:

will be announced later